

Atelier — Planètes (orbites)

À ouvrir après 1-COURS.pdf du dossier projets/02-orbites/.

1–3 périodes. Quatre planètes. Ce n'est pas le grand projet.

Le dossier `depart/` de cet exercice contient les fichiers de départ. Vous n'avez pas à ouvrir un terminal.

Où l'on va

Vous allez reprendre Mercure, Terre, Saturne et Neptune, les quatre objets des fiches. Cette fois, ils ne restent pas immobiles : chaque objet apprend à avancer et à se dessiner. Le Soleil et les orbites forment seulement le décor de cet exercice.

À la fin, votre écran ressemble à ceci. Saturne se reconnaît, le bouton Pause est visible et un choix affiche le nom et le type d'une planète.



Sur un téléphone de 390 pixels de large, le canvas se resserre. Le bouton Pause reste lisible et aucun mot n'est coupé.



Vous créez exactement ceci

```
sas/  
├─ orbites/  
│  └─ index.html  
│     └─ css/  
│        └─ style.css  
│           └─ js/  
│              └─ planete.js
```

Le départ contient déjà le HTML et le CSS. Votre travail se fait dans `js/planete.js` : vous complétez la classe, vous créez les objets, puis vous les faites travailler ensemble.

Étape 1 — Ouvrir le départ

Dans le premier exercice, les fiches étaient quatre objets immobiles : elles affichaient des données, mais elles n'agissaient pas. Ici, les mêmes quatre objets apprennent un métier de plus. Ils vont

pouvoir modifier leur angle et tracer leur propre disque.

Ouvrez `sas/orbites/index.html` par un double-clic. Vous devez voir une page couleur nuit, un canvas vide et le bouton Pause déjà présent. Le vide est normal : aucune planète n'a encore été fabriquée.

- ☐ L'arbre correspond exactement au dessin ci-dessus.
- ☐ Le fond est sombre.
- ☐ Le canvas est vide.
- ☐ Le bouton Pause est visible.

Si la page est blanche, vérifiez que `index.html` charge bien `css/style.css`. Vous n'avez pas à recopier la page ni son style depuis ce PDF.

Étape 2 — Fabriquer un seul objet

Imaginez qu'il faille remplir à la main les mêmes sept cases pour chaque planète, puis recommencer au moindre changement. Une **classe** évite cette répétition : c'est le moule. `new Planete(...)` utilise ce moule pour produire **un objet réel**, comme un moule à billets produit un billet réel.

Ouvrez `js/planete.js`. Le constructeur du départ reçoit six valeurs et les range sur l'objet avec `this`. Il fixe aussi `angle` à 0 : toute nouvelle planète part donc au même endroit.

```
constructor(nom, distance, rayon, couleur, vitesse, type) {  
  this.nom = nom;  
  this.distance = distance;  
  this.rayon = rayon;  
  this.couleur = couleur;  
  this.vitesse = vitesse;  
  this.angle = 0;  
  this.type = type;  
}
```

Sous la classe, fabriquez seulement la Terre :

```
const terre = new Planete(  
  "Terre", 125, 11, "#3d8fd1", 0.018, "rocheuse"  
);
```

Ouvrez les outils de développement du navigateur, puis la console. Tapez `terre.nom`, puis `terre.angle`. Vous devez lire "Terre", puis 0. Le mot `terre` désigne cet objet précis ; `Planete` reste le moule.

Étape 3 — L'objet avance

Si la page augmente elle-même la propriété `angle`, la planète reste un sac de chiffres que le reste du

fichier manipule. Une **méthode** est une action que l'objet sait faire lui-même. Ici, l'objet connaît déjà son angle et sa vitesse : c'est donc lui qui doit avancer.

Dans la méthode `avancer()` du squelette, écrivez la règle **ajouter la vitesse à l'angle**. Cette règle tient sur une ligne :

```
this.angle += this.vitesse
```

Ne placez pas cette ligne sous la classe. Elle doit remplacer le commentaire `/* à vous : une ligne */` **dans** `avancer()`.

Rechargez la page. Dans la console, appelez deux fois la méthode, puis relisez l'angle :

```
terre.avancer();
terre.avancer();
terre.angle;
```

Vous devez obtenir environ `0.036`. Vous venez de parler à l'objet : vous lui avez demandé deux fois d'avancer.

Étape 4 — L'objet trouve sa position et se dessine

Un cercle tracé dans une grande fonction ailleurs dans le fichier n'appartient plus vraiment à la planète. La méthode `dessiner(ctx, cx, cy)` vit donc **dans** la classe. Le centre du canvas se passe avec `cx` et `cy`, car il appartient à la scène ; l'angle et la distance restent sur l'objet, car c'est lui qui les connaît.

4.1 La règle de position

Complétez d'abord `position(cx, cy)`. La règle nommée **position sur un cercle** est :

```
x = cx + cos(angle) * distance
y = cy + sin(angle) * distance
```

En JavaScript, la méthode doit retourner cet objet :

```
return {
  x: cx + Math.cos(this.angle) * this.distance,
  y: cy + Math.sin(this.angle) * this.distance,
};
```

Cette méthode est sur la planète parce que la planète connaît `this.angle` et `this.distance`. La boucle n'aura pas à chercher `x` et `y` elle-même.

4.2 Le disque de la planète

Complétez maintenant `dessiner(ctx, cx, cy)` sans déplacer la méthode hors de la classe. Écrivez les cinq gestes suivants dans l'ordre :

- 1 demander la position avec `this.position(cx, cy)` et la garder dans `lieu` ;
- 2 commencer un chemin avec `ctx.beginPath()` ;
- 3 choisir `this.couleur` dans `ctx.fillStyle` ;
- 4 tracer un cercle centré sur `lieu.x`, `lieu.y`, de rayon `this.rayon` ;
- 5 remplir ce cercle.

Voici le squelette à compléter. Les deux lignes centrales restent volontairement à écrire :

```
dessiner(ctx, cx, cy) {
  const lieu = this.position(cx, cy);
  ctx.beginPath();
  /* à vous : choisir la couleur de remplissage */
  /* à vous : ctx.arc avec lieu.x, lieu.y et this.rayon */
  ctx.fill();
}
```

Pour `ctx.arc`, un cercle complet commence à 0 et finit à `Math.PI * 2`.

Ajoutez ensuite ce test temporaire sous `const terre = ...` :

```
const canvas = document.getElementById("ciel");
const ctx = canvas.getContext("2d");
const cx = canvas.width / 2;
const cy = canvas.height / 2;
terre.dessiner(ctx, cx, cy);
```

Rechargez. Vous devez voir **une seule Terre**, à droite du centre. Si le canvas reste vide, vérifiez la console : la première erreur indique souvent la ligne incomplète.

Étape 5 — Quatre objets, une boucle mince

Vous avez fabriqué une Terre avec le moule. Les quatre fiches utilisent maintenant ce même moule : seules les valeurs changent. La boucle ne calcule pas leur position ; elle demande à chaque objet d'avancer, puis de se dessiner.

Remplacez `const terre = ...` par les quatre objets suivants. Gardez les nombres : ils règlent la cible de l'exercice.

Nom	distance	rayon	couleur	vitesse	type
Mercure	70	7	#b7b3ab	0.032	"rocheuse"
Terre	125	11	#3d8fd1	0.018	"rocheuse"
Saturne	190	18	#e6c36a	0.008	"gazeuse"
Neptune	240	13	#4169c9	0.005	"gazeuse"

```
const systeme = [
  new Planete("Mercure", 70, 7, "#b7b3ab", 0.032, "rocheuse"),
  new Planete("Terre", 125, 11, "#3d8fd1", 0.018, "rocheuse"),
  new Planete("Saturne", 190, 18, "#e6c36a", 0.008, "gazeuse"),
  new Planete("Neptune", 240, 13, "#4169c9", 0.005, "gazeuse"),
];
```

Gardez les quatre lignes `canvas`, `ctx`, `cx`, `cy` du test. Supprimez seulement l'appel temporaire `terre.dessiner(...)`.

Le décor n'est pas une planète

Ajoutez cette petite fonction sous les objets. Elle efface l'image précédente, puis trace le Soleil. Le Soleil reste le décor fixe de cet exercice ; il n'a pas besoin du moule `Planete`.

```
function dessinerDecor() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.beginPath();
  ctx.fillStyle = "#f5c16c";
  ctx.arc(cx, cy, 22, 0, Math.PI * 2);
  ctx.fill();
}
```

La boucle demande, les objets font

Ajoutez enfin la boucle. Son cœur tient en quelques lignes : elle ne contient ni formule de position, ni `ctx.arc` pour une planète.

```
let enPause = false;

function boucle() {
  dessinerDecor();
  systeme.forEach((planete) => {
    if (!enPause) planete.avancer();
    planete.dessiner(ctx, cx, cy);
  });
  requestAnimationFrame(boucle);
}

boucle();
```

Rechargez. Vous devez voir quatre disques qui tournent. Au premier instant, ils partent tous à droite parce que leur angle vaut 0, puis leurs vitesses les séparent. À ce stade, un disque qui tourne suffit : les anneaux de Saturne viendront après.

Étape 6 — Pause, puis parler à un objet

Les objets tournent ; il reste à leur parler sans fouiller dans leurs chiffres. `estGazeuse()` reprend une règle déjà rencontrée avec les fiches : l'objet répond lui-même à la question. Le bouton Pause, lui, ne modifie pas la classe ; il change seulement l'état de la scène.

6.1 Règle nommée : planète gazeuse

Dans `estGazeuse()`, écrivez la règle **le type vaut "gazeuse"** :

```
return this.type === "gazeuse"
```

6.2 Règle nommée : mouvement réduit

Remplacez `let enPause = false;` par ces deux lignes. Si la personne a demandé moins de mouvement dans son système, la page démarre en pause.

```
const prefereMoins = window.matchMedia("(prefers-reduced-motion: reduce)").matches;
let enPause = prefereMoins;
```

Ajoutez le comportement du bouton :

```
const boutonPause = document.getElementById("pause");

function etiquettePause() {
  boutonPause.textContent = enPause ? "Reprendre" : "Pause";
}

etiquettePause();
boutonPause.addEventListener("click", () => {
  enPause = !enPause;
  etiquettePause();
});
```

6.3 Règle nommée : afficher le type de l'objet

Les quatre boutons de noms sont déjà dans le HTML. Ajoutez ce comportement court : un clic, ou le focus suivi d'Entrée, trouve l'objet puis lui demande s'il est gazeux.

```
const fiche = document.getElementById("fiche");

document.querySelectorAll("#noms button").forEach((bouton) => {
  bouton.addEventListener("click", () => {
    const planete = systeme.find((objet) => objet.nom === bouton.dataset.nom);
    if (!planete) return;
    const reponse = planete.estGazeuse() ? "oui" : "non";
    fiche.textContent = `${planete.nom} – gazeuse : ${reponse}.`;
  });
});
```

Rechargez. Pause arrête les objets ; Reprendre les relance. Cliquez sur Terre, puis sur Saturne. Vous devez lire gazeuse : non, puis gazeuse : oui. Avec le clavier, placez le focus sur un bouton et appuyez sur Entrée.

Réussi si

- [] La classe est le moule et vous savez montrer un objet créé avec `new Planete(...)`.

- [] Les quatre objets existent, avec les données du tableau.
- [] `avancer()` et `dessiner()` sont dans la classe.
- [] `position(cx, cy)` applique la règle du cercle sur l'objet.
- [] La boucle ne calcule ni `x` ni `y`.
- [] Le bouton affiche Pause ou Reprendre.
- [] Avec `prefers-reduced-motion: reduce`, la page démarre en pause.
- [] Un clic, ou focus puis Entrée, affiche le nom et gazeuse oui/non via `estGazeuse()`.
- [] La page se lit à 1440 et à 390 pixels de large.

AVEC L'IA

Vous avez écrit `avancer()` et `dessiner()` vous-même. Montrez maintenant seulement votre boucle à une IA et demandez : « Expliquez en cinq phrases qui calcule la position et pourquoi la boucle n'écrit ni `x` ni `y`. » Comparez sa réponse avec votre classe ; ne lui demandez pas de refaire le système solaire.

POUR LES RAPIDES

Ajoutez une méthode `reglerVitesse(v)` dans la classe. Elle remplace `this.vitesse` par `v`. Testez-la sur un des quatre objets depuis la console : vous modifiez ainsi le comportement à travers la classe, sans ajouter une cinquième planète.

Quand un disque tourne correctement, vous pouvez aussi dessiner deux ellipses autour de Saturne **dans** `dessiner()`. Les anneaux servent à la reconnaître ; ils ne changent pas le concept de classe, d'objet ou de méthode.

Livrable

Rendez le dossier `sas/orbites/` avec ses trois fichiers. La page doit s'ouvrir par double-clic. À l'oral, vous montrez un `new Planete(...)`, puis vous expliquez pourquoi `avancer()` et `dessiner()` sont dans la classe.